Task 20 — Data Types and Type Conversion

Description: Inspect DataFrame data types and convert columns between strings, integers, floats, Booleans and dates.

Objective: Understand how correct data types affect data processing and analysis.

Tools: Python, Pandas, Jupyter Notebook.

1. Learning Objectives

By the end of this task, you should be able to inspect the data types of DataFrame columns, decide which types match the meaning of each column, convert strings into numbers, convert text labels into Boolean values, parse date strings, detect values that failed conversion, and save a separate cleaned dataset.

Practical outcome: a raw passenger-style CSV, a converted CSV, a data-type report, a Python script, a notebook, and this guide.

2. Data Types in Simple Language

A data type is the kind of value a column contains. A person's name is text; age is normally a whole number; a fare may have decimals; a yes/no flag is Boolean; and a travel date is datetime.

Type is not just how a value looks. The string '25' contains two characters, while the integer 25 is a number. The string '2026-01-05' looks like a date, but it is still text until parsed as a datetime value.

3. Why the Same-Looking Values Behave Differently

Text concatenation: '2' + '3' produces '23' when both values are strings. Numeric addition: 2 + 3 produces 5. This is one reason to check types before calculations.

Sorting text values may produce an order such as '1', '10', '2', while numeric sorting produces 1, 2, 10. Dates represented inconsistently as text can also sort incorrectly.

4. Main Data Types and Typical Uses

String/object: names, cities, ticket classes and identifiers that are not used in arithmetic.

Integer: counts, ages in whole years, number of items, and 0/1 codes when integer semantics are desired.

Float: fares, prices, measurements and other decimal values.

Boolean: true/false flags such as IsStudent, IsActive, or HasPaid.

Datetime: dates and timestamps used for time-based filtering, grouping and calculations.

Category: repeated labels from a limited set, such as Economy, Business and First. A category can save memory and make the meaning of the field explicit.

5. Inspect Before Converting

Use `df.head()` to see example rows, `df.shape` to see the row/column count, `df.dtypes` to see each type, and `df.info()` to see non-null counts and types.

Do not convert a column only because its name sounds numeric. Inspect actual values and decide what the column means. PassengerId, for example, is an identifier: arithmetic on it usually has no useful interpretation.

6. Understand df.dtypes

Command: `print(df.dtypes)`. The output lists one dtype per column. If Age is `object` or `string`, although it should represent a number, inspect it for values such as 'unknown', 'NA', or spaces.

After conversion, Age may show nullable `Int64`, Fare may show `float64`, IsStudent may show nullable `boolean`, and TravelDate may show `datetime64[ns]`.

7. Understand df.info()

Command: `df.info()`. It shows column names, non-null counts, dtypes and approximate memory use. If a DataFrame has 12 rows but Age has only 10 non-null values after conversion, two Age entries are missing or invalid.

`info()` is an overview; it does not explain why values are missing. Use filters and `isna()` to investigate specific rows.

8. The Role of astype()

`astype()` explicitly changes the representation of compatible values. Example: `df['Name'] = df['Name'].astype('string')` makes the text intent explicit.

`astype('int64')` can fail if a value is 'unknown' or missing. Inspect and clean values first, or use a parsing method that can mark invalid values as missing.

9. The Role of pd.to_numeric()

`pd.to_numeric(series)` parses values into numbers. Example: `pd.to_numeric(df['Fare'], errors='coerce')` converts '120.50' into 120.50.

When `errors='coerce'` is used, strings that cannot be parsed become NaN. This is a controlled way to continue, but you must inspect the failed values because coercion does not fix them.

10. Understanding errors='coerce'

For numeric conversion, invalid entries become `NaN`. For datetime conversion, invalid dates become `NaT`. These markers tell you parsing failed or data is missing.

Do not assume every invalid token means zero. For example, 'free' could mean a genuinely free ticket or could be a data-entry error. Ask what the field means before replacing it.

11. Convert Age Carefully

Age is normally a whole-number measurement. In this dataset, values such as '22' and ' 28 ' can become integers after trimming spaces. Values such as 'unknown' and 'NA' cannot be parsed as ages and should become missing for review.

Example: `df['Age'] = pd.to_numeric(df['Age'].astype(str).str.strip().replace({'NA': None, 'unknown': None}), errors='coerce').astype('Int64')`.

Nullable `Int64` (capital I) allows both integer values and missing values. Standard NumPy `int64` does not directly support NaN in the same way.

12. Convert Fare Carefully

Fare may contain decimal values, so `float64` is appropriate. `pd.to_numeric()` converts strings such as '120.50' and '99.90'.

An entry such as 'free' becomes missing under coercion. Before changing that missing value to 0, confirm whether 'free' truly means a zero fare in the source system.

13. Convert a 0/1 Outcome

Survived is represented as 1 or 0 in this example. Converting it to nullable `Int64` preserves the numeric code and permits missing values.

If a downstream task specifically expects a Boolean target, it may be converted to True/False after confirming that only 0 and 1 are present. Never assume arbitrary numbers can safely become a Boolean target.

14. Convert Text to Boolean

Use an explicit mapping: `df['IsStudent'].map({'True': True, 'False': False})`. This translates known labels rather than treating any non-empty string as True.

If a source contains 'yes', 'no', 'Y', 'N', '1' or '0', define a mapping for the expected values and inspect any unmapped entries.

15. Convert TravelDate

Use `pd.to_datetime(df['TravelDate'], errors='coerce')`. Valid strings become datetime values and can be sorted chronologically, filtered by date, or grouped by month.

An entry such as 'invalid-date' becomes `NaT`. Inspect the original value before deciding whether it can be corrected.

16. What Are NaN, None and NaT?

`NaN` is commonly used for missing numeric values. `None` is Python's null-like object and can be used when preparing data for conversion. `NaT` represents missing or invalid datetime values.

Use `isna()` to detect missing values regardless of which missing marker is used in a Pandas column. Use `notna()` to find values that are present.

17. Check What Failed to Convert

After conversion, use `df.isna().sum()` to count missing values per column. To inspect rows with missing Age, use `df[df['Age'].isna()]`.

Important: a missing value after conversion may have existed in the source, or it may have been created because a value such as 'unknown' could not be parsed. Keep the raw column or compare with the raw dataset if you need to distinguish these cases.

18. Before-and-After Type Report

The report file compares original dtype, converted dtype and missing values after conversion. This documents the transformation and helps another person review what changed.

Example expected types: Age → Int64; Fare → float64; Survived → Int64; IsStudent → boolean; TravelDate → datetime64[ns]; Name and TicketClass → text/string.

19. Clean Data Does Not Mean No Missing Values

A successful conversion can still leave missing values. The goal is not to hide missingness; the goal is to represent data correctly and make unresolved values visible.

Next steps depend on the context: correct a source typo, ask the data owner, impute a value with a justified method, exclude a row for a specific analysis, or preserve the missing value.

20. Common Conversion Errors

ValueError with `astype(int)`: one or more values cannot be parsed as integers. Inspect the values and handle invalid entries first.

Unexpected NaN: `errors='coerce'` may have converted invalid strings to missing values. Review the original raw values.

Unexpected Boolean missing values: a text label was not included in the mapping dictionary.

Date parsing issue: the date format may be inconsistent or ambiguous. Check examples and, where needed, specify the expected format.

21. Validate After Conversion

Check that Age contains plausible values for the task, Fare is not negative if negative fares are not allowed, Survived contains only 0 or 1, IsStudent contains True/False or missing, and TravelDate contains parseable dates or documented missing values.

Type conversion and validation are related but different. Conversion changes representation; validation checks whether the resulting values obey the rules of the dataset.

22. A Safer End-to-End Workflow

1. Preserve the raw source. 2. Inspect rows and dtypes. 3. Identify the intended type of each column. 4. Normalize spaces and known placeholders. 5. Convert values. 6. Count and inspect conversion failures. 7. Validate ranges and categories. 8. Save a separate cleaned dataset. 9. Document the rules.

23. Complete Example With Comments

Load the CSV, print original types, convert numeric columns, map the Boolean column, parse dates, print converted types, count missing values, and save the cleaned file.

Run `Task_20.py` from the folder containing `passenger_raw.csv`. If using Jupyter, run the notebook cells from top to bottom.

24. Practice Exercises

Exercise 1: Create a Series containing ['10', '20', 'unknown'] and convert it to numeric using `errors='coerce'`. How many missing values result?

Exercise 2: Convert ['True', 'False', 'True'] to Boolean values using `map()`.

Exercise 3: Convert ['2026-01-01', 'bad-date'] using `pd.to_datetime(..., errors='coerce')`. What is the invalid value converted to?

Exercise 4: Inspect a DataFrame using `dtypes`, `info()` and `isna().sum()` and explain what each command tells you.

25. Practice Exercise Answers

Answer 1: The first two values become 10 and 20; 'unknown' becomes NaN, so one value is missing.

Answer 2: The values become True, False and True, assuming the strings exactly match the mapping keys.

Answer 3: The valid date becomes datetime; 'bad-date' becomes NaT.

Answer 4: `dtypes` lists column types; `info()` gives structural details and non-null counts; `isna().sum()` counts missing values in each column.

26. Interview Questions and Answers

Q: Why are correct data types important? A: They allow appropriate operations and improve reliability.

Q: What does astype() do? A: Casts compatible values to a specified dtype.

Q: How do you convert strings to numeric? A: Use `pd.to_numeric()` and inspect values that become NaN.

Q: How do you parse dates? A: Use `pd.to_datetime()` and inspect values that become NaT.

Q: Why use nullable Int64? A: It supports whole-number values and missing values in the same column.

Q: Is type conversion the same as validation? A: No. Conversion changes representation; validation checks whether values satisfy rules.

27. Top 5 Skills Used

1. Pandas. 2. Data Type Inspection. 3. Type Conversion. 4. Data Cleaning. 5. Python Data Analysis.

28. Feedback

The task helped me understand how data types affect data processing and analysis. I learned to inspect DataFrame types, convert numeric strings, map text labels to Boolean values, parse dates, and investigate missing values created during conversion. The practical dataset strengthened my Pandas and data-cleaning skills.

Complete Python Program

```
import pandas as pd

df = pd.read_csv("passenger_raw.csv")

print("ORIGINAL TYPES")
print(df.dtypes)

df["Age"] = pd.to_numeric(
    df["Age"].astype(str).str.strip().replace({"NA": None, "unknown": None}),
    errors="coerce"
).astype("Int64")

df["Fare"] = pd.to_numeric(
    df["Fare"].astype(str).str.strip().replace({"NA": None}),
    errors="coerce"
)

df["Survived"] = pd.to_numeric(df["Survived"], errors="coerce").astype("Int64")
df["IsStudent"] = df["IsStudent"].map(
    {"True": True, "False": False}
).astype("boolean")
df["TravelDate"] = pd.to_datetime(df["TravelDate"], errors="coerce")
df["TicketClass"] = df["TicketClass"].astype("string")

print("\\nCONVERTED TYPES")
print(df.dtypes)
print("\\nMISSING VALUES")
print(df.isna().sum())

df.to_csv("passenger_cleaned.csv", index=False)
```

Quick Reference Table

Purpose	Method	What it does
Inspect types	df.dtypes	Lists dtype for each column

Inspect structure	df.info()	Shows non-null counts and dtypes
Convert compatible values	astype()	Casts to a requested type
Parse numbers	pd.to_numeric()	Converts numeric strings
Parse dates	pd.to_datetime()	Converts date strings
Handle parsing failures	errors='coerce'	Invalid values become missing
Count missing data	df.isna().sum()	Counts missing values by column